

Algoritma

- Cari daun dengan $f(n)$ terendah
- Coba bergerak dalam setiap arah dari posisi awal dalam daun, hasil yang valid ditambah sebagai daun
- Ulang sampai dapat solusi atau semua daun adalah dead-end (hanya bisa kembali ke posisi parent)

$$f(n) = g(n) + h(n)$$

$g(n)$ = total biaya tile yang sudah dijalani pemain

heuristik:

Manhattan distance: admissible*

Cherbyshev distance: inadmissible

Euclidean distance: admissible*

*secara umum

Perbandingan

Manhattan > Euclidean > Cherbyshev

Alternatif: Dijkstra dari goal ke posisi awal, manhattan dimodifikasi ($h=0$ jika bisa lurus ke goal, $h=1$ jika bisa ke goal dalam 2 input, $h=2$ untuk kasus lain)

Kode

```
#ifndef STR_UTILS_H
#define STR_UTILS_H

int strtol(const char *data, char **endptr, int base);

#endif

#include <errno.h>
#include <stdlib.h>
#include <limits.h>
#include "str_utils.h"

int strtol(const char *data, char **endptr, int base)
{
    int old_errno = errno;
    errno = 0;
    long lval = strtol(data, endptr, base);
    if (lval > INT_MAX)
    {
        errno = ERANGE;
        lval = INT_MAX;
    }
    else if (lval < INT_MIN)
    {
        errno = ERANGE;
        lval = INT_MIN;
    }
    if (errno == 0)
        errno = old_errno;
    return (int)lval;
}

#ifndef GAME_STATE_H
#define GAME_STATE_H

#include <stdio.h>

#define MAX_ROWS 100
#define MAX_COLS 100

typedef enum {
    PATH,
    OBSTACLE,
    LAVA,
    GOAL
} tile_type;

typedef struct {
    tile_type **tiles;
    int **costs;
    signed char **goals;
    int rows;
    int cols;
    signed char max_goal;
} game_board;
```

```

typedef struct {
    double heuristic;
    double total;
    int row;
    int col;
    int cost;
    signed char next_goal;
} player_state;

char create_board(game_board *board, player_state *player);
void free_board(game_board *board);
void print_board(game_board *board, player_state *player);
void print_board_to_file(game_board *board, player_state *player, FILE *fp);
#endif

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include "game_state.h"
#include "str_utils.h"

char create_board(game_board *board, player_state *player) {
    char input[101];
    int cur_row = 0;
    player->row = -1;
    player->col = -1;
    player->next_goal = 10;
    player->heuristic = 1;
    player->cost = 0;
    player->total = 0;
    board->max_goal = -1;

    fputs("Enter board dimensions: ", stdout);

    while (fgets(input, sizeof(input), stdin)) {
        char *end;
        board->rows = strtol(input, &end, 10);
        if (end == input + strlen(input) - 1) {
            fputs("Only one number detected. Try again: ", stdout);
            continue;
        }
        board->cols = strtol(end, &end, 10);
        if (end == input) {
            fputs("No numbers detected. Try again: ", stdout);
        } else if (end != input + strlen(input) - 1) {
            fputs("Invalid input. Try again: ", stdout);
        } else if (board->rows <= 0 || board->rows > MAX_ROWS || board->cols <= 0 || board->cols > MAX_COLS) {
            fputs("Inputs out of range. Try again: ", stdout);
        } else {
            break;
        }
    }

    board->tiles = (tile_type**)malloc(board->rows * sizeof(tile_type));
    if (board->tiles == NULL) {
        return 0;
    }
    board->costs = (int**)malloc(board->rows * sizeof(int));
    if (board->costs == NULL) {
        free_board(board);
        return 0;
    }
    board->goals = (signed char**)malloc(board->rows * sizeof(signed char));
    if (board->goals == NULL) {
        free_board(board);
        return 0;
    }
    for (int i = 0; i < board->rows; i++) {
        board->tiles[i] = (tile_type*)malloc(board->cols * sizeof(tile_type));
        if (board->tiles[i] == NULL) {
            free_board(board);
            return 0;
        }
        board->costs[i] = (int*)malloc(board->cols * sizeof(int));
        if (board->costs[i] == NULL) {
            free_board(board);
            return 0;
        }
        board->goals[i] = (signed char*)malloc(board->cols * sizeof(signed char));
        if (board->goals[i] == NULL) {
            free_board(board);
            return 0;
        }
    }

    for (int i = 0; i < board->rows; i++) {
        for (int j = 0; j < board->cols; j++) {
            board->tiles[i][j] = OBSTACLE;
            board->costs[i][j] = 999999;
            board->goals[i][j] = -1;
        }
    }

    puts("Enter board state, allowed chars: *, X, L, Z, O, 0-9");
    while (cur_row < board->rows && fgets(input, sizeof(input), stdin)) {
        fputs("\n", stdout);
        if (strlen(input) - 1 != board->cols) {
            puts("Input row doesn't match number of board columns");
            continue;
        }

        for (int i = 0; i < board->cols; i++) {
            switch (input[i]) {
                case '*':
                    board->tiles[cur_row][i] = PATH;
                    break;
                case 'x':
                case 'X':
                    board->tiles[cur_row][i] = OBSTACLE;
                    break;
                case 'l':
                case 'L':
                    board->tiles[cur_row][i] = LAVA;
                    break;
                case 'z':
                case 'Z':
                    if (player->row == -1 && player->col == -1) {
                        player->row = cur_row;
                        player->col = i;
                        board->tiles[cur_row][i] = PATH;
                    } else {
                        puts("Multiple player symbols detected.");
                        i--;
                    }
                    break;
                case 'o':
                case 'O':
                    board->tiles[cur_row][i] = GOAL;
                    break;
                default:
                    if (input[i] >= '0' && input[i] <= '9') {
                        board->goals[cur_row][i] = input[i] - '0';
                        board->tiles[cur_row][i] = PATH;
                        if (player->next_goal > input[i] - '0') player->next_goal = input[i] - '0';
                        if (board->max_goal < input[i] - '0') board->max_goal = input[i] - '0';
                    } else {
                        puts("Invalid character detected.");
                        i--;
                    }
                    break;
            }
        }
        cur_row++;
    }
}

```

```

        }
        cur_row++;
    }

    cur_row = 0;
    puts("Enter board costs");

    while (cur_row < board->rows && fgetc(input, sizeof(input), stdin)) {
        fputc(" ", stdout);
        char *end = input;
        for (int i = 0; i < board->cols; i++) {
            char *end2;
            board->costs[cur_row][i] = strtol(end, &end2, 10);
            if (end == end2) {
                fputc("Invalid input.", stdout);
                cur_row--;
                break;
            }
            end = end2;
        }
        cur_row++;
    }

    return 1;
}

void free_board(game_board *board) {
    for (int i = 0; i < board->rows; i++) {
        if (board->tiles[i] != NULL) {
            free(board->tiles[i]);
            board->tiles[i] = NULL;
        }
        if (board->costs[i] != NULL) {
            free(board->costs[i]);
            board->costs[i] = NULL;
        }
        if (board->goals[i] != NULL) {
            free(board->goals[i]);
            board->goals[i] = NULL;
        }
    }
    if (board->tiles != NULL) {
        free(board->tiles);
        board->tiles = NULL;
    }
    if (board->costs != NULL) {
        free(board->costs);
        board->costs = NULL;
    }
    if (board->goals != NULL) {
        free(board->goals);
        board->goals = NULL;
    }
}

void print_board(game_board *board, player_state *player) {
    for (int i = 0; i < board->rows; i++) {
        for (int j = 0; j < board->cols; j++) {
            if (player->row == i && player->col == j) {
                fputc("Z", stdout);
                continue;
            }
            if (board->goals[i][j] >= 0) {
                printf("%d", board->goals[i][j]);
                continue;
            }
            switch (board->tiles[i][j]) {
                case PATH:
                    fputc(" ", stdout);
                    break;
                case OBSTACLE:
                    fputc("X", stdout);
                    break;
                case LAWA:
                    fputc("L", stdout);
                    break;
                case GOAL:
                    fputc("O", stdout);
                    break;
            }
        }
        puts("");
    }
}

void print_board_to_file(game_board *board, player_state *player, FILE *fp) {
    for (int i = 0; i < board->rows; i++) {
        for (int j = 0; j < board->cols; j++) {
            if (player->row == i && player->col == j) {
                fputc("Z", fp);
                continue;
            }
            if (board->goals[i][j] >= 0) {
                fprintf(fp, "%d", board->goals[i][j]);
                continue;
            }
            switch (board->tiles[i][j]) {
                case PATH:
                    fputc(" ", fp);
                    break;
                case OBSTACLE:
                    fputc("X", fp);
                    break;
                case LAWA:
                    fputc("L", fp);
                    break;
                case GOAL:
                    fputc("O", fp);
                    break;
            }
        }
        fputc('\n', fp);
    }
}

#ifdef GAME_SOLVER_H
#define GAME_SOLVER_H

#include "game_state.h"

typedef enum {
    LEFT,
    UP,
    DOWN,
    RIGHT,
    NONE
} direction;

typedef enum {
    MANHATTAN,
    PYTHAGOREAN,
    CHERBSYSHEV
} heuristic_functions;

typedef struct tr* tree_address;
typedef struct tr {
    player_state player;
    tree_address child[4];
    tree_address parent;
    direction last_dir;
    direction solution;
    char is_leaf;
} tree;

tree* create_node(tree *par, player_state *pl, direction dir);

```

```

double get_heuristic(game_board *board, player_state *pl, heuristic_functions hf);

player_state try_sliding(game_board *board, player_state *pl, direction dir);

void expand(game_board *board, tree *t, heuristic_functions hf);

tree* find_lowest_cost_leaf(tree* t);

void set_solution_path(tree *t);

int Astar(game_board *board, tree *root, heuristic_functions hf);

void print_tree(tree *t, int depth);

void free_tree(tree *t);

int get_max_depth(tree *t, int starting_depth);

#endif

#include "game_solver.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

tree* create_node(tree *par, player_state *pl, direction dir) {
    tree *t = (tree*)malloc(sizeof(tree));
    if (t == NULL) { return NULL;
    }
    t->parent = par;
    t->player = *pl;
    t->last_dir = dir;
    for (int i = 0; i < 4; i++) {
        t->child[i] = NULL;
    }
    t->solution = NONE;
    t->is_leaf = 1;
    return t;
}

double get_heuristic(game_board *board, player_state *pl, heuristic_functions hf) {
    switch (hf) {
        case MANHATTAN:
            double mh = board->rows + board->cols;
            for (int i = 0; i < board->rows; i++) {
                for (int j = 0; j < board->cols; j++) {
                    if ((pl->next_goal < 10 && board->goals[i][j] == pl->next_goal) || (pl->next_goal == 10 && board->tiles[i][j] ==
GOAL)) {
                        int temp = abs(i - pl->row) + abs(j - pl->col);
                        if (mh > temp) mh = temp;
                    }
                }
            }
            return mh;
            break;
        case PYTHAGOREAN:
            double py = board->rows + board->cols;
            for (int i = 0; i < board->rows; i++) {
                for (int j = 0; j < board->cols; j++) {
                    if ((pl->next_goal < 10 && board->goals[i][j] == pl->next_goal) || (pl->next_goal == 10 && board->tiles[i][j] ==
GOAL)) {
                        double temp = sqrt((i - pl->row) * (i - pl->row) + (j - pl->col) * (j - pl->col));
                        if (py > temp) py = temp;
                    }
                }
            }
            return py;
            break;
        case CHERBYSHEV:
            double ch = board->rows + board->cols;
            for (int i = 0; i < board->rows; i++) {
                for (int j = 0; j < board->cols; j++) {
                    if ((pl->next_goal < 10 && board->goals[i][j] == pl->next_goal) || (pl->next_goal == 10 && board->tiles[i][j] ==
GOAL)) {
                        int temp = abs(i - pl->row);
                        if (abs(j - pl->col) > temp) temp = abs(j - pl->col);
                        if (ch > temp) ch = temp;
                    }
                }
            }
            return ch;
            break;
    }
}

player_state try_sliding(game_board *board, player_state *pl, direction dir) {
    puts("k");
    player_state p = *pl;
    signed char horz = 0;
    signed char vert = 0;
    switch (dir) {
        case LEFT:
            horz = -1;
            vert = 0;
            break;
        case RIGHT:
            horz = 1;
            vert = 0;
            break;
        case DOWN:
            horz = 0;
            vert = 1;
            break;
        case UP:
            horz = 0;
            vert = -1;
            break;
    }
    if (p.row + vert < 0 || p.row + vert >= board->rows ||
        p.col + horz < 0 || p.col + horz >= board->cols ||
        board->tiles[p.row + vert][p.col + horz] == LAVA ||
        board->goals[p.row + vert][p.col + horz] > p.next_goal) {
        p.row = -1;
        p.col = -1;
        return p;
    }
    while (board->tiles[p.row + vert][p.col + horz] != OBSTACLE) {
        if (p.row + vert < 0 || p.row + vert >= board->rows ||
            p.col + horz < 0 || p.col + horz >= board->cols ||
            board->tiles[p.row + vert][p.col + horz] == LAVA ||
            board->goals[p.row + vert][p.col + horz] > p.next_goal) {
            p.row = -1;
            p.col = -1;
            break;
        }
        if (board->goals[p.row + vert][p.col + horz] == p.next_goal) p.next_goal++;
        if (p.next_goal > board->max_goal) p.next_goal = 10;
        p.row += vert;
        p.col += horz;
        p.cost += board->costs[p.row][p.col];
        if (p.row + vert < 0 || p.row + vert >= board->rows ||
            p.col + horz < 0 || p.col + horz >= board->cols ||
            board->tiles[p.row + vert][p.col + horz] == LAVA ||
            board->goals[p.row + vert][p.col + horz] > p.next_goal) {
            p.row = -1;
            p.col = -1;
            break;
        }
    }
    if (pl->row == p.row && pl->col == p.col) {
        p.row = -1;
        p.col = -1;
    }
}

```

```

        puts("j");
        return p;
    }

void expand(game_board *board, tree *t, heuristic_functions hf) {
    puts("d");
    t->is_leaf = 0;
    char flag = 0;
    for (int i = 0; i < 4; i++) {
        player_state pl = try_sliding(board, &(t->player), i);
        //if (t->parent != NULL && pl.row == t->parent->player.row && pl.col == t->parent->player.col) continue;
        if (pl.row != -1 && pl.col != -1) {
            flag = 1;
            t->child[i] = create_node(t, &pl, i);
            t->child[i]->player.heuristic = get_heuristic(board, &pl, hf);
            t->child[i]->player.total = t->child[i]->player.heuristic + t->child[i]->player.cost;
        }
    }
    if (flag == 0) t->is_leaf = 2;
}

tree* find_lowest_cost_leaf(tree* t) {
    if (t->is_leaf == 1) {
        return t;
    } else {
        tree *out = NULL;
        int total = INT_MAX;
        signed char best_goal = -1;
        //printf("%d %d %d %f %f %d\n", t->player.row, t->player.col, t->player.cost, t->player.heuristic, t->player.total, t->player.next_goal, t->is_leaf);
        for (int i = 0; i < 4; i++) {
            if (t->child[i] == NULL) continue;
            //printf("%d %d %d %f %f %d\n", t->child[i]->player.row, t->child[i]->player.col, t->child[i]->player.cost, t->child[i]->player.heuristic, t->child[i]->player.total, t->child[i]->player.next_goal, t->child[i]->is_leaf);
            if (t->child[i]->is_leaf == 1) {
                if (t->child[i]->player.next_goal > best_goal) {
                    best_goal = t->child[i]->player.next_goal;
                    total = t->child[i]->player.total;
                    out = t->child[i];
                } else if (t->child[i]->player.total < total && t->child[i]->player.next_goal == best_goal) {
                    total = t->child[i]->player.total;
                    out = t->child[i];
                }
            } else if (t->child[i]->is_leaf == 0) {
                tree *temp = find_lowest_cost_leaf(t->child[i]);
                if (temp != NULL) {
                    if (temp->player.next_goal > best_goal) {
                        best_goal = temp->player.next_goal;
                        total = temp->player.total;
                        out = temp;
                    } else if (temp->player.total < total && temp->player.next_goal == best_goal) {
                        total = temp->player.total;
                        out = temp;
                    }
                }
            }
        }
        return out;
    }
}

void set_solution_path(tree *t) {
    if (t->parent != NULL) {
        t->parent->solution = t->last_dir;
        set_solution_path(t->parent);
    }
}

int Astar(game_board *board, tree *root, heuristic_functions hf) {
    int i = 0;
    while (i < 9999999) {
        *cur_working_node = find_lowest_cost_leaf(root);
        if (cur_working_node == NULL) {
            i = -1;
            break;
        }
        printf("%d %d %d %f %f %d\n", cur_working_node->player.row, cur_working_node->player.col, cur_working_node->player.cost, cur_working_node->player.heuristic, cur_working_node->player.total, cur_working_node->player.next_goal);
        if (cur_working_node->player.heuristic <= 0.000001) {
            set_solution_path(cur_working_node);
            break;
        }
        expand(board, cur_working_node, hf);
        print_tree(root, 0);
        i++;
    }
    if (i >= 9999999) i = -1;
    return i;
}

void print_tree(tree *t, int depth) {
    printf("%s%d %d %d %f %f %d\n", depth, "", t->player.row, t->player.col, t->player.cost, t->player.heuristic, t->player.total, t->player.next_goal, t->is_leaf);
    for (int i = 0; i < 4; i++) {
        if (t->child[i] != NULL) print_tree(t->child[i], depth + 1);
    }
}

void free_tree(tree *t) {
    for (int i = 0; i < 4; i++) {
        if (t->child[i] != NULL) {
            free_tree(t->child[i]);
            t->child[i] = NULL;
        }
    }
    free(t);
}

int get_max_depth(tree *t, int starting_depth) {
    int m = starting_depth;
    for (int i = 0; i < 4; i++) {
        if (t->child[i] != NULL) {
            int tt = get_max_depth(t->child[i], starting_depth + 1);
            if (tt > m) m = tt;
        }
    }
    return m;
}

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "game_state.h"
#include "game_solver.h"

static void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    // Assume POSIX (Linux, macOS, etc.)
    system("clear");
#endif
}

int main() {
    game_board board;
    player_state player;

    if (create_board(&board, &player) != 0) {
        tree *root = create_node(NULL, &player, NONE);
        heuristic_functions hf = MANHATTAN;
        int hch = -1;
        do {
            printf("Input heuristic function[0-2]: ");
            scanf("%d", &hch);
            while (getchar() != '\n');
        } while (hch < 0 || hch > 2);
    }
}

```

```

switch (hch) {
    case 0:
        hf = MANHATTAN;
        break;
    case 1:
        hf = CHERBYSEV;
        break;
    case 2:
        hf = PYTHAGOREAN;
        break;
}
clock_t start_time = clock();
int n_of_its = Astar(&board, root, hf);
if (n_of_its == -1) {
    puts("Solution not found");
    exit(EXIT_SUCCESS);
}
clock_t end_time = clock();

tree *tnode = root;

int tctr = 0;

while (tnode->is_leaf == 0) {
    char cc;
    switch (tnode->last_dir) {
        case LEFT:
            cc = 'L';
            break;
        case RIGHT:
            cc = 'R';
            break;
        case UP:
            cc = 'U';
            break;
        case DOWN:
            cc = 'D';
            break;
    }
    printf("Step %d, Input:%c\n", tctr, cc);
    print_board(&board, &(tnode->player));
    puts("");
    tctr++;
    tnode = tnode->child[tnode->solution];
}

char ccc = 0;
switch (tnode->last_dir) {
    case LEFT:
        ccc = 'L';
        break;
    case RIGHT:
        ccc = 'R';
        break;
    case UP:
        ccc = 'U';
        break;
    case DOWN:
        ccc = 'D';
        break;
}
printf("Step %d, Input:%c\n", tctr, ccc);
print_board(&board, &(tnode->player));
puts("");

fputs("Solution: ", stdout);
tnode = root;
while (tnode->is_leaf == 0) {
    switch (tnode->last_dir) {
        case LEFT:
            fputs("L", stdout);
            break;
        case RIGHT:
            fputs("R", stdout);
            break;
        case UP:
            fputs("U", stdout);
            break;
        case DOWN:
            fputs("D", stdout);
            break;
    }
    tnode = tnode->child[tnode->solution];
}
switch (tnode->last_dir) {
    case LEFT:
        fputs("L", stdout);
        break;
    case RIGHT:
        fputs("R", stdout);
        break;
    case UP:
        fputs("U", stdout);
        break;
    case DOWN:
        fputs("D", stdout);
        break;
}

printf("\nJumlah iterasi: %d\n", n_of_its);
printf("Cost: %d\n", tnode->player.cost);
printf("Elapsed time: %.2lf ms\n\n", (double)(end_time - start_time) / CLOCKS_PER_SEC * 1000);
fputs("Playback? (y/n): ", stdout);
int yn;
char flag = 0;
yn = getchar();
if (yn == 'y' || yn == 'Y') flag = 1;

tree *cur_node = root;

int cur_depth = 0;
while (flag == 1) {
    int in;
    clearScreen();
    print_board(&board, &(cur_node->player));
    fputs("Input: ", stdout);
    in = getchar();
    switch (in) {
        case ',':
            if (cur_node->parent != NULL) {
                cur_node = cur_node->parent;
                cur_depth--;
            }
            break;
        case '.':
            if (cur_node->is_leaf == 0) {
                cur_node = cur_node->child[cur_node->solution];
                cur_depth++;
            }
            break;
        case 'q':
            flag = 0;
            break;
        case 'e':
            int step = -1;
            do {
                printf("Input step: ");
                scanf("%d", &step);
                while (getchar() != '\n');
            } while (step < 0 || step > get_max_depth(root, 0));
            while (cur_depth > step) {
                cur_node = cur_node->parent;
                cur_depth--;
            }
            while (cur_depth < step) {
                cur_node = cur_node->child[cur_node->solution];
                cur_depth++;
            }
            break;
    }
}
}

```

```

    )
    FILE *outptr;
    outptr = fopen("test/output.txt", "w");
    fprintf(outptr, "\xEF\xBB\xBF");
    tree *tempnode = root;
    int tempctr = 0;
    while (tempnode->is_leaf == 0) {
        char cc;
        switch (tempnode->last_dir) {
            case LEFT:
                cc = 'L';
                break;
            case RIGHT:
                cc = 'R';
                break;
            case UP:
                cc = 'U';
                break;
            case DOWN:
                cc = 'D';
                break;
        }
        fprintf(outptr, "Step %d, Input:%c\n", tempctr, cc);
        print_board_to_file(&bboard, &(tempnode->player), outptr);
        fputc('\n', outptr);
        tempctr++;
        tempnode = tempnode->child[tempnode->solution];
    }
    char cc = 0;
    switch (tempnode->last_dir) {
        case LEFT:
            cc = 'L';
            break;
        case RIGHT:
            cc = 'R';
            break;
        case UP:
            cc = 'U';
            break;
        case DOWN:
            cc = 'D';
            break;
    }
    fprintf(outptr, "Step %d, Input:%c\n", tempctr, cc);
    print_board_to_file(&bboard, &(tnode->player), outptr);
    fputc('\n', outptr);
    fclose(outptr);
    free_tree(root);
}

free_board(&bboard);
return 0;
}

```

Step 4, Input:R

X**X

X*X0X

X*L*X

X1*ZX

XXXXX

Solution: RDLR

Jumlah iterasi: 4

Cost: 9

Elapsed time: 4.00 ms

```
Step 8, Input:R
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

Solution: RULUDRUR
Jumlah iterasi: 11
Cost: 87
Elapsed time: 11.00 ms
```

```
Step 7, Input:R
XX*XXXXX
X*****0X
X**X**XX
X*****ZX
X***X*XX
XXXXXXXXX

Solution: URLDRUR
Jumlah iterasi: 26
Cost: 80
Elapsed time: 37.00 ms
```

```
Step 13, Input:R
*X**X**
*1X2**X*
****X****
X*****X
*X**0**ZX
****X*X**

Solution: DULUDRULULRDR
Jumlah iterasi: 20
Cost: 42
Elapsed time: 27.00 ms
```

```
Step 8, Input:L
XXXXXXXXX
X*****X
X*XX*L*X
*****X
X0XXZ**
XXXXXXXXX

Solution: RULDURDL
Jumlah iterasi: 24
Cost: 125
Elapsed time: 28.00 ms
```

Analisis

Ada pilihan lebih baik dari jarak ke goal dalam pemilihan heuristik untuk ice sliding puzzle

Pranala

Github: https://github.com/kalkabena/Tucil3_13524119.git

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Nathanael Shane Bennet